



Lecture 3 – Logistic Regression

Presented By

Md. Shoab Alam (Lecturer)

Green University of Bangladesh

Department of CSE

Regression	Classification
Supervised Machine Learning	Supervised Machine Learning
Predictions of Labels OR Target Variables Which are Continuous	Predictions of Labels OR Target Variables Which are Discrete
Examples are Salaries, Housing Price	Classification of disease OR Classification of Animals
Housing Price = 952340.98076	Yes/No, Cat/Dog

Logistic Regression

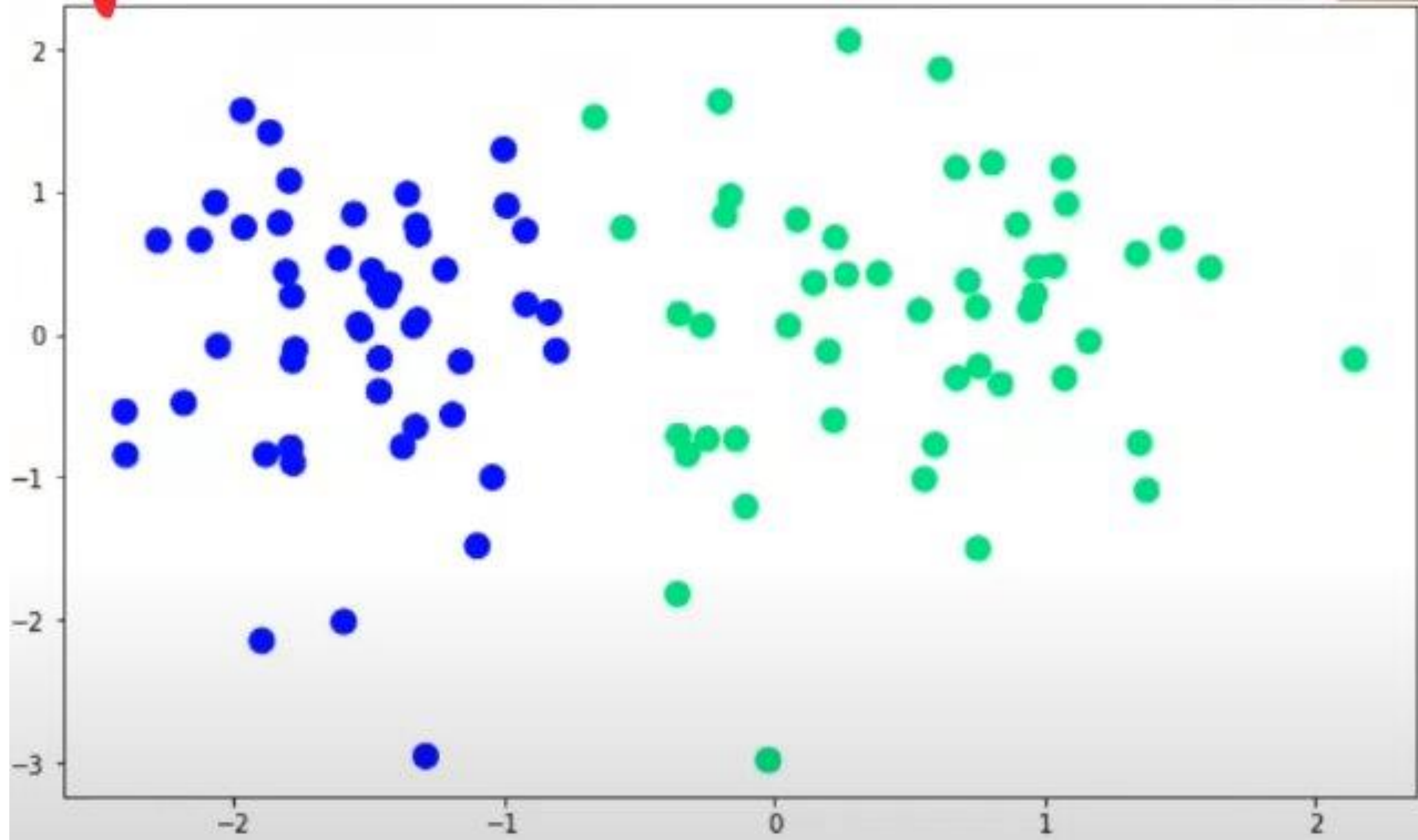
A Logistic Regression (also known as binary logistic regression) has logical target variables OR Labels.

- Yes / No
- Pass / Fail
- Male / Female

Types of Logistic Regression

Logistic regression can be classified into three main types based on the nature of the dependent variable:

1. **Binomial Logistic Regression:** This type is used when the dependent variable has only two possible categories. Examples include Yes/No, Pass/Fail or 0/1. It is the most common form of logistic regression and is used for binary classification problems.
2. **Multinomial Logistic Regression:** This is used when the dependent variable has three or more possible categories that are not ordered. For example, classifying animals into categories like "cat," "dog" or "sheep." It extends the binary logistic regression to handle multiple classes.
3. **Ordinal Logistic Regression:** This type applies when the dependent variable has three or more categories with a natural order or ranking. Examples include ratings like "low," "medium" and "high." It takes the order of the categories into account when modeling.



Perception Trick

1. Formula of line = $Ax_1 + Bx_2 + C = 0$
2. Perception algorithm
 1. Select a random A B C
 2. Loop for 1000 or till convergence
 1. Select a random point, check if the line properly classifying or not.
 1. If yes skip, select another
 2. If no, Calculate such A,B,C so that the line correctly classifies the values

Transformation of A,B,C (Desmos)

1

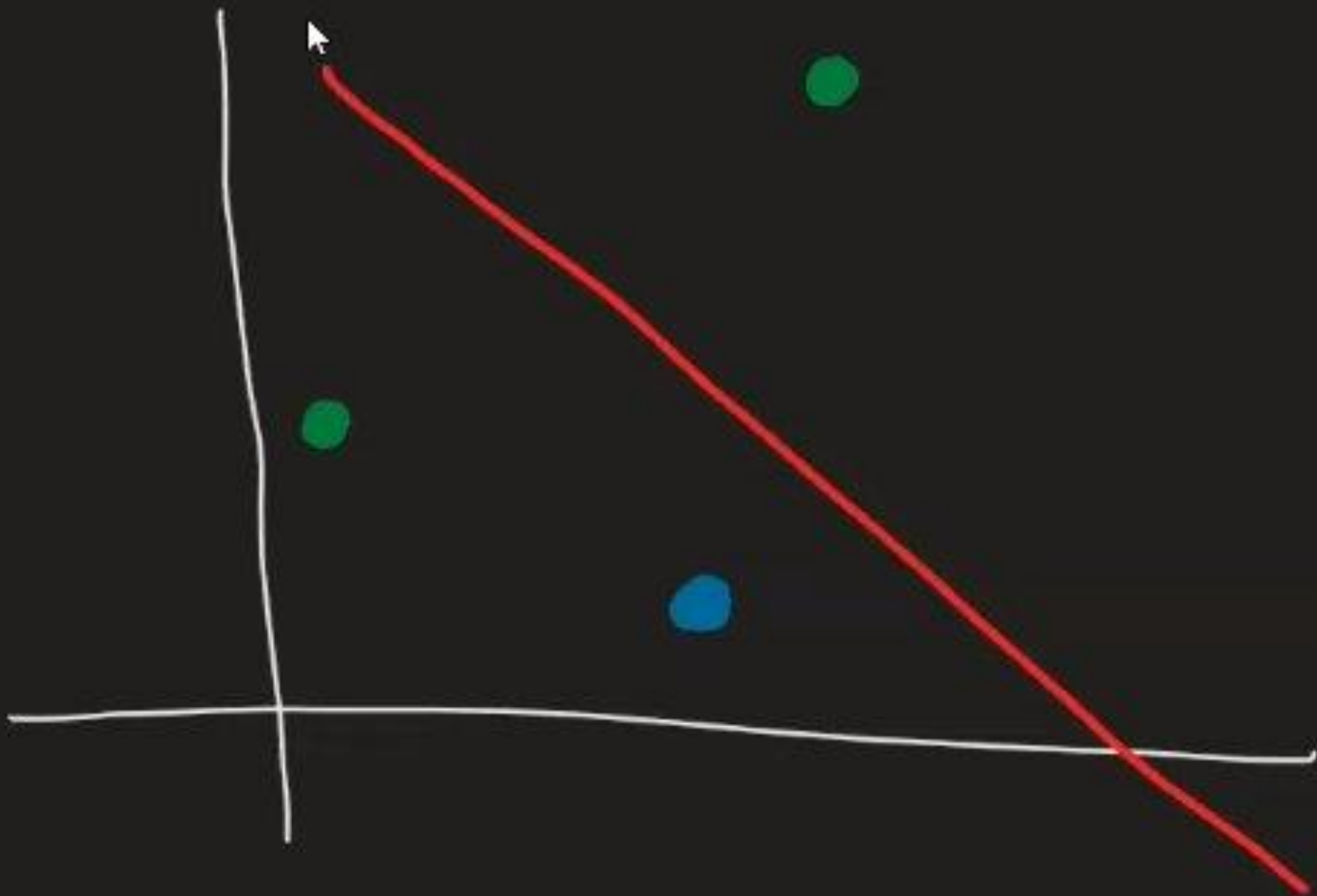
Changes of C, [C higher, Line Lower vice versa]

2

Increase in A, fixed in y, x will change clockwise vice versa.

3

Increase in B, fixed in x, y will change anti clockwise vice versa.



RULE

Transformation Formula

1

If a negative point is in positive point,

1. Add 1 at last in the coordinate's values
2. Subtract coordinate's values from lines coefficient

2

If a positive point in negative point,

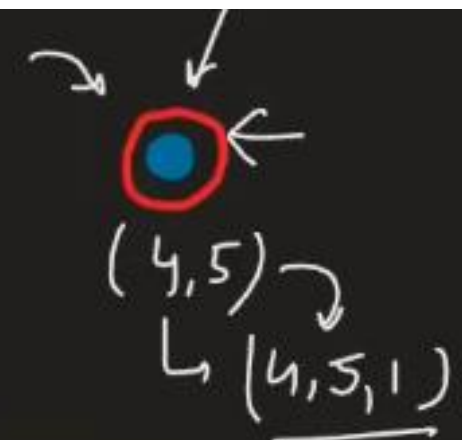
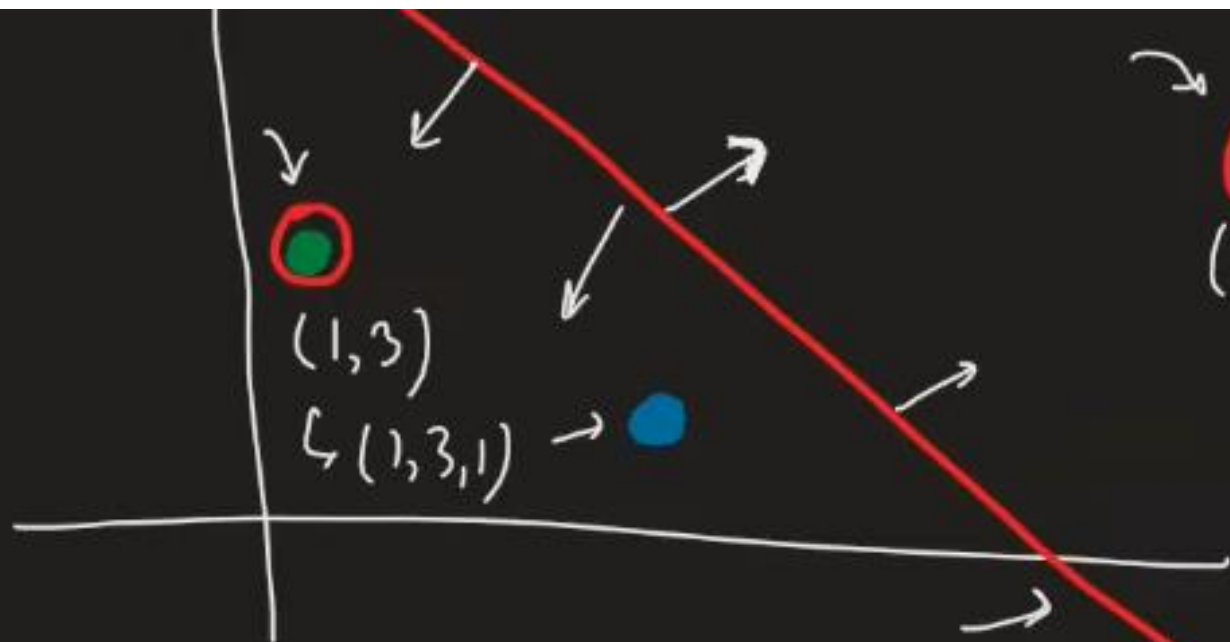
1. Add 1 at last in the coordinate's values
2. Add coordinate's values with the lines coefficient

1. For both sub, add we multiply a **learning rate** with data points

2. Formula

$$\text{New_coeff} = \text{old_coeff} - n * \text{coordinates}$$

This makes the changes gradually.



$$\begin{array}{r}
 2 \quad 3 \quad 5 \\
 4 \quad 5 \quad 1 \\
 \hline
 -2x - 2y + 4 = 0
 \end{array}$$

$$\underline{2x + 3y + 5 = 0}$$

blue

$$\begin{array}{r}
 2 \quad 3 \quad 5 \\
 1 \quad 3 \quad 1 \\
 \hline
 + \quad 3x + 6y + 6 = 0
 \end{array}$$

WORKED EXAMPLE

Line Coefficient $\rightarrow$ Prediction

x0	CGPA (x1)	iq (x2)	Placement
1	7.5	81	1
1	9.5	109	1
1	8.5	60	0

line coefficient $\rightarrow \mathbf{Ax1 + Bx2 + c = 0}$

$\rightarrow \mathbf{w0} + \mathbf{w1x1} + \mathbf{w2x2} = 0 \quad (C, A, B)$

$\rightarrow w0x0 + w1x1 + w2x2 = 0$

$\therefore \Sigma w_i x_i = 0$

$\Rightarrow$ **Prediction**

$\rightarrow 1 \quad 7.5 \quad 81$

$$w0 \times 1 + w1 \times 7.5 + w2 \times 81 = 0$$

if $\mathbf{>0} \rightarrow \mathbf{1}$

$\mathbf{<0} \rightarrow \mathbf{0}$

ALGORITHM

Algorithm for Perception Trick

```
1. Epoch 1000,  $\eta = 0.01$ 
```

```
    Select a random A, B, C
```

```
    for i in range(epochs):
```

```
        Select a random student[i]
```

```
            If  $X_i \in N$  and sum of  $W_i X_i \geq 0$ 
```

```
                 $W_{new} = W_{old} - \eta * X_i$ 
```

```
            If  $X_i \in P$  and sum of  $W_i X_i < 0$ 
```

```
                 $W_{new} = W_{old} + \eta * X_i$ 
```

Algorithm for Perception Trick

Perception Trick

Formula of line = $Ax_1 + Bx_2 + C = 0$

Perception algorithm – epoch =1000, $n=0.01$ (learning rate)

Select a random A B C

For in i in range(epochs):

Select a random student(i),

$$\mathbf{W}_n = \mathbf{W}_0 + n \cdot (y_i - y_i^{\wedge}) \cdot \mathbf{X}_i$$

Where $\mathbf{w}_0$ is the current weights and $\mathbf{x}_i$ is the sample input.

WHY IT WORKS

How the Formula Fits 4 Scenarios

The error term ($y_i - \hat{y}_i$) decides what the update does — add, subtract, or do nothing.

y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	Update
1	1	0	no change
0	0	0	no change
1	0	+1	$W0 + n \cdot X_i$
0	1	-1	$W0 - n \cdot X_i$

Plugging the error into $W_n = W_0 + n \cdot (y_i - \hat{y}_i) \cdot X_i$:

0 Correct positive
 $y_i=1, \hat{y}_i=1$

No change — the point is already classified correctly, so weights stay put.

0 Correct negative
 $y_i=0, \hat{y}_i=0$

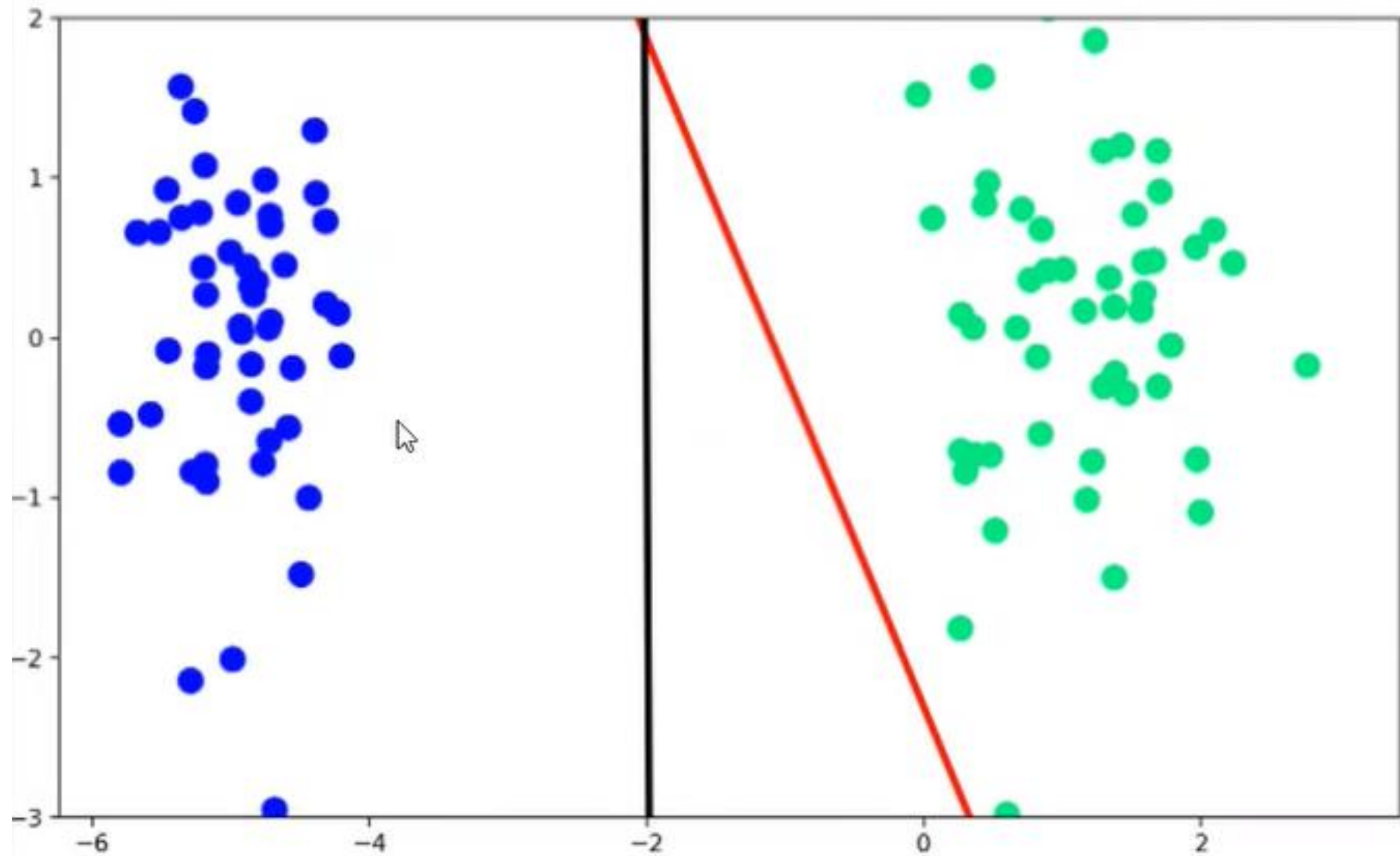
No change — again correct, the formula multiplies by 0 and does nothing.

+1 Missed positive
 $y_i=1, \hat{y}_i=0$

Positive point predicted negative
→ ADD $n \cdot X_i$ to push the line the right way.

-1 Missed negative
 $y_i=0, \hat{y}_i=1$

Negative point predicted positive
→ SUBTRACT $n \cdot X_i$ to correct the line.



Problem With Perceptron

1. Perceptron only updates on misclassified points — ignores correctly classified ones entirely.
2. Once all points are correct, learning stops regardless of boundary quality.
3. Correctly classified points near the boundary have zero influence — same as far-away points
4. **Logistic regression fixes this** — every point contributes via probability, pushing for higher confidence even when correctly classified

Logistic Regression

1. Old algorithm: only misclassified points affect the line — correctly classified are ignored
2. New algorithm: every point contributes — misclassified pulls, correctly classified pushes
3. Magnitude of push/pull depends on distance from the boundary
4. Misclassified + Far $\rightarrow$ Large pull — model is confidently wrong, highest error, strongest correction
5. Misclassified + Close $\rightarrow$ Moderate pull — model is uncertain and wrong, still needs fixing
6. Correctly classified + Close $\rightarrow$ Moderate push — model is correct but barely, boundary still shaky
7. Correctly classified + Far $\rightarrow$ Small push — model is confident and correct, minimal adjustment needed

WHY IT WORKS

How the Formula Fits 4 Scenarios

The error term ($y_i - \hat{y}_i$) decides what the update does — add, subtract, or do nothing.

y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	Update
1	1	0	no change
0	0	0	no change
1	0	+1	$W0 + n \cdot X_i$
0	1	-1	$W0 - n \cdot X_i$

Plugging the error into $W_n = W_0 + n \cdot (y_i - \hat{y}_i) \cdot X_i$:

0 Correct positive
 $y_i=1, \hat{y}_i=1$

No change — the point is already classified correctly, so weights stay put.

0 Correct negative
 $y_i=0, \hat{y}_i=0$

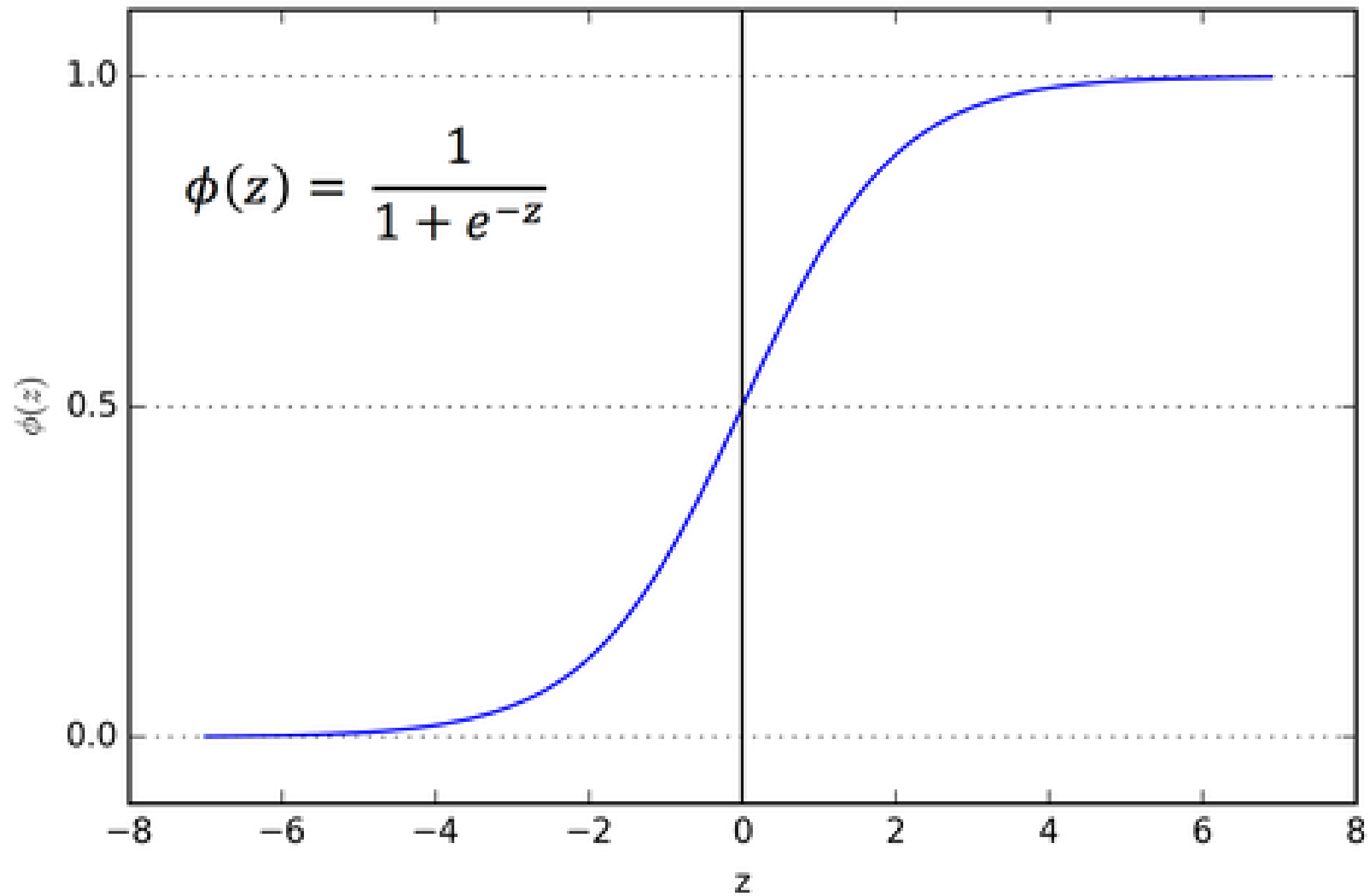
No change — again correct, the formula multiplies by 0 and does nothing.

+1 Missed positive
 $y_i=1, \hat{y}_i=0$

Positive point predicted negative
→ ADD $n \cdot X_i$ to push the line the right way.

-1 Missed negative
 $y_i=0, \hat{y}_i=1$

Negative point predicted positive
→ SUBTRACT $n \cdot X_i$ to correct the line.



Impact of Sigmoid

Thursday, June 17, 2021 3:27 PM

$$w_{\eta} = w_0 + \eta (y_i - \hat{y}_i) x_i$$

$$\hat{y}_i = \sigma(z)$$

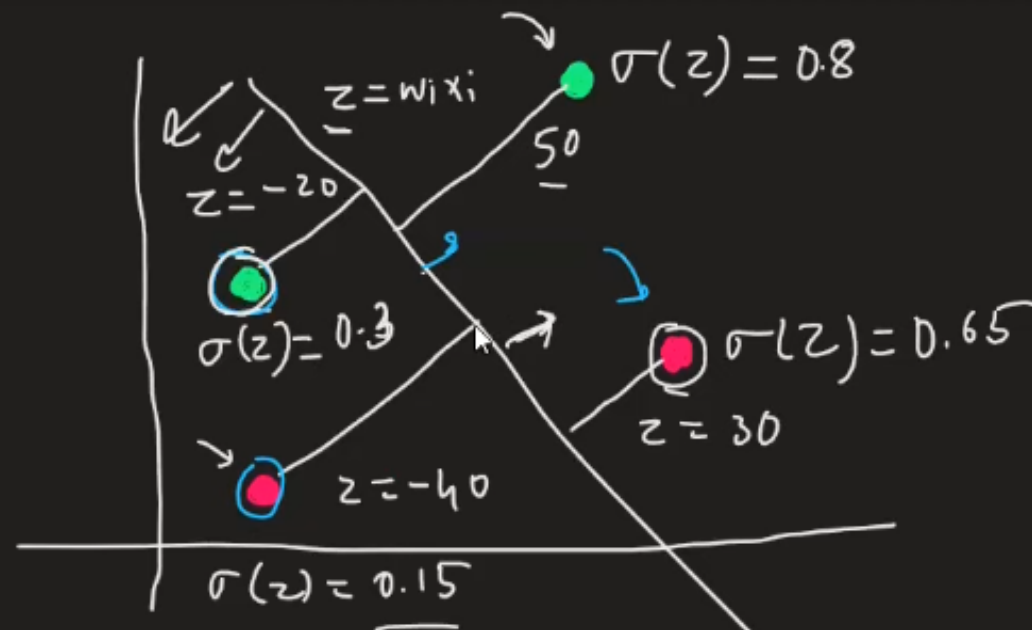
$$\text{where } z = \sum w_i x_i$$

$$w_{\eta} = \underline{w_0} + \eta \times 0.2 \times x_i$$

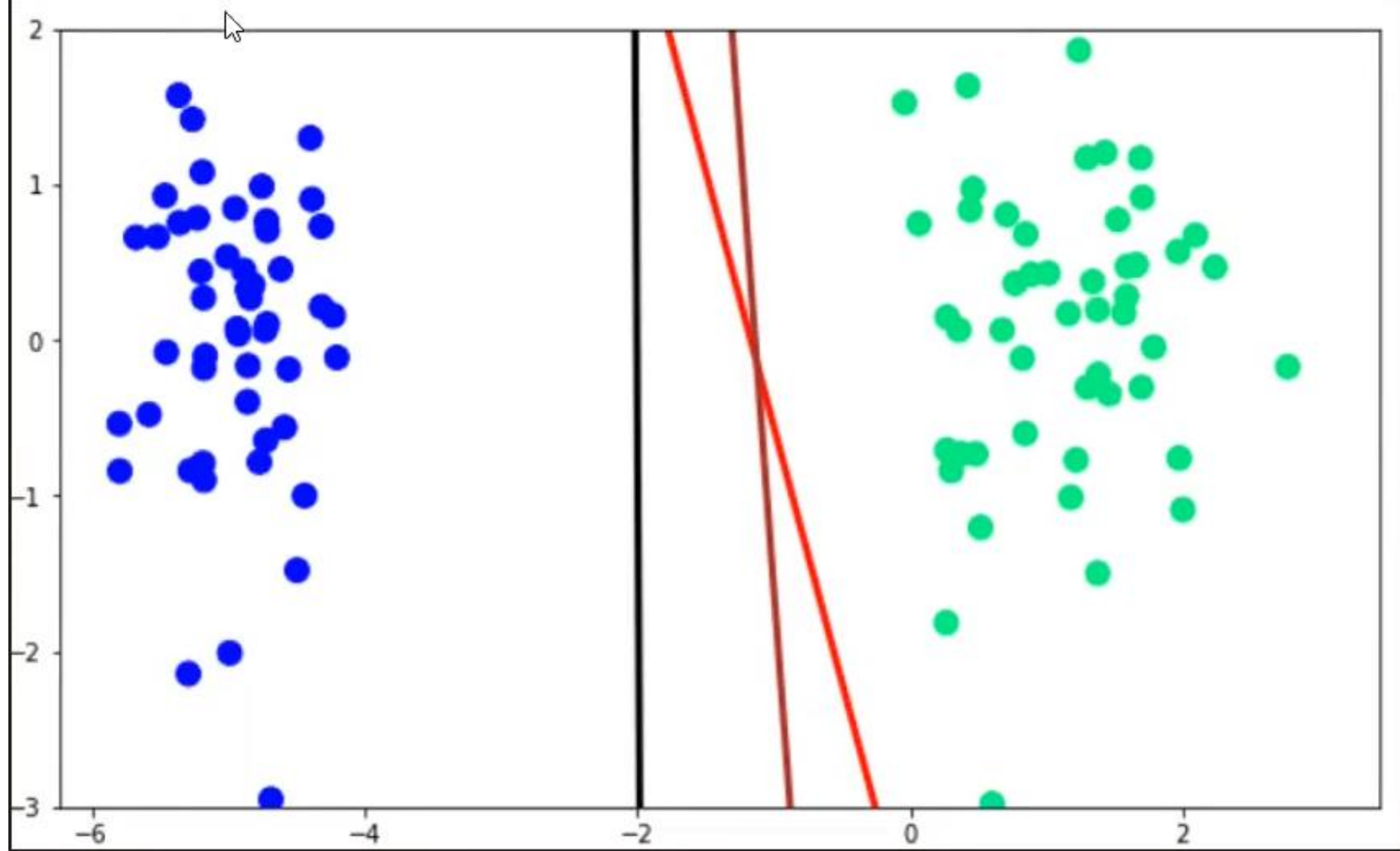
$$w_{\eta} = \underline{w_0} - \eta \times 0.65 \times x_i$$

$$w_{\eta} = w_0 + \eta \times 0.7 \times x_i$$

$$w_{\eta} = \underline{w_0} - \eta \times 0.15 \times x_i$$

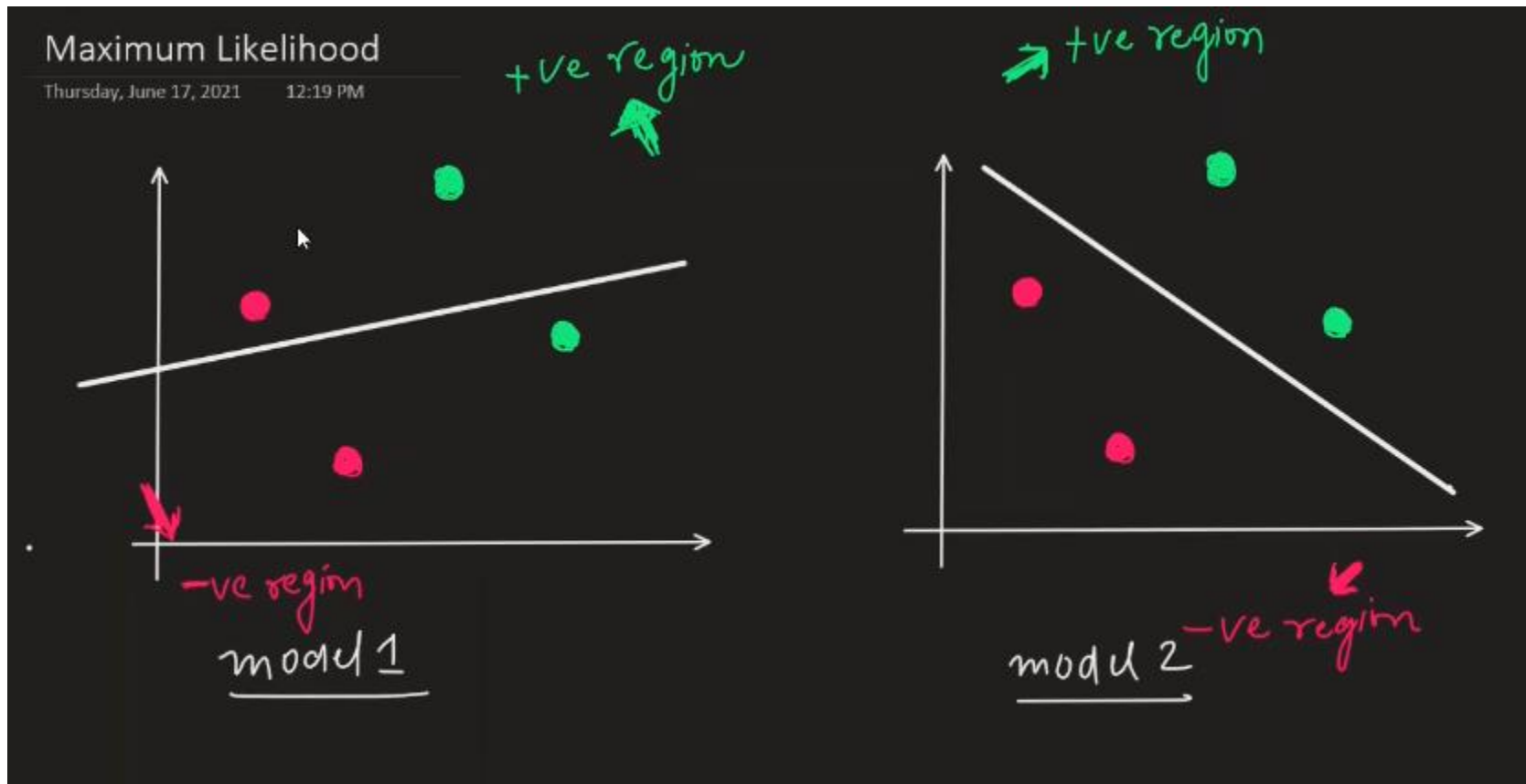


x_i	$\hat{y}_i$	$y_i - \hat{y}_i$
1	0.8	0.2
0	0.65	-0.65
1	0.3	0.7
0	0.15	-0.15



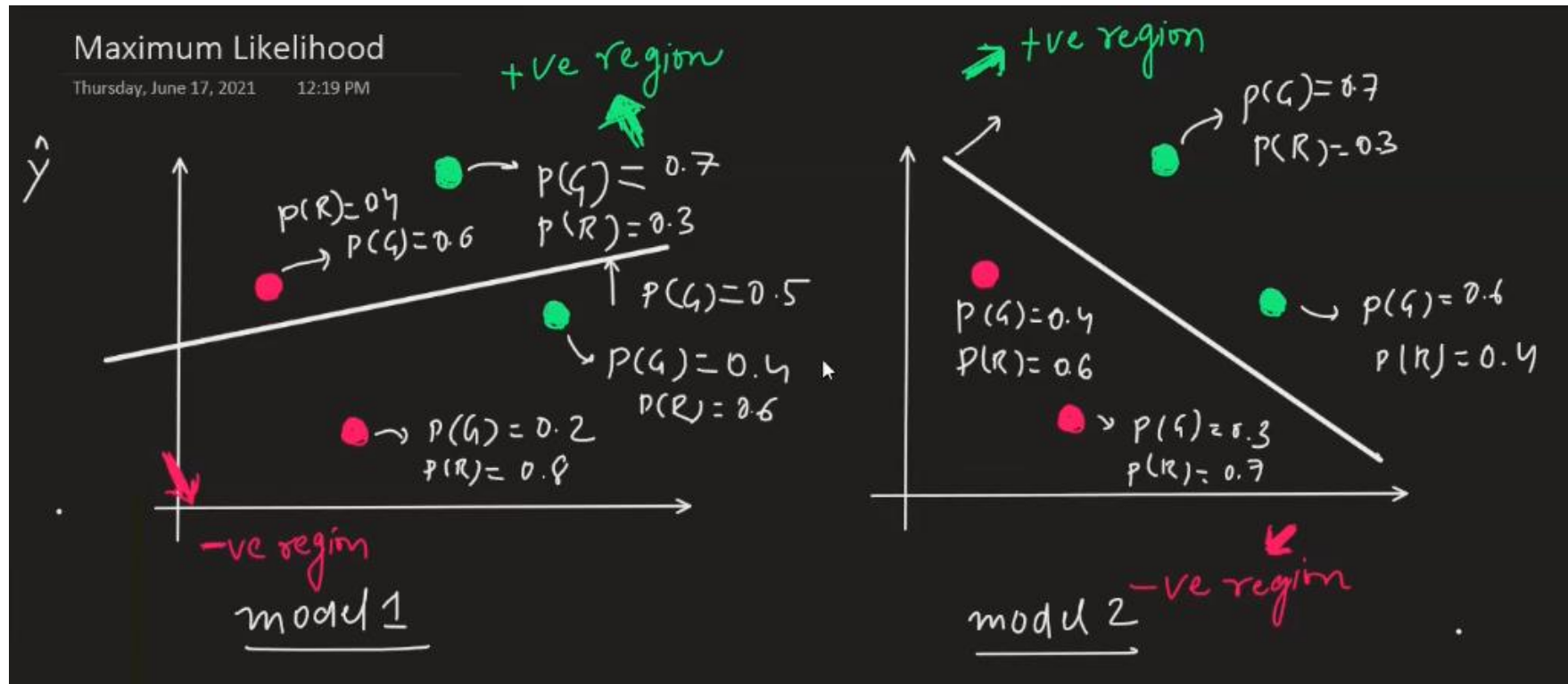
Loss Function

1. A loss function is a measure of how wrong your model is. It takes the model's prediction and the true label, and outputs a single number — the penalty for being wrong.



Maximum Likelihood

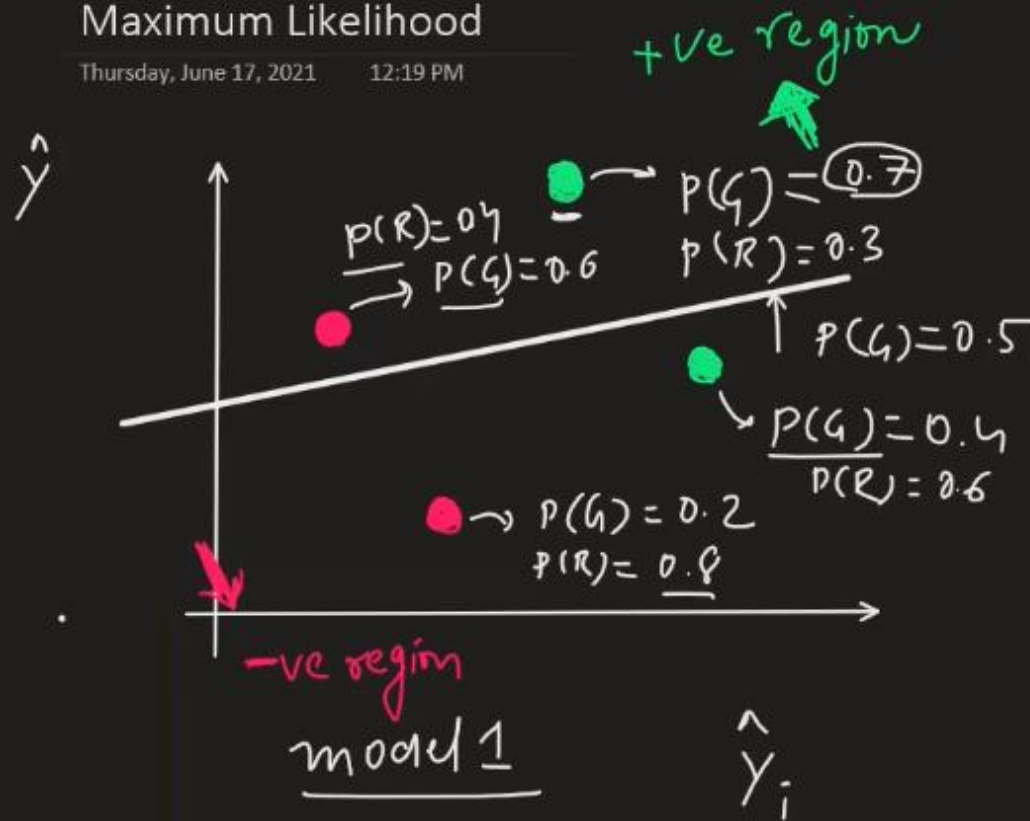
1. Maximum Likelihood finds model coefficients that maximize the probability of observing the actual training data.
2. The total likelihood is the product of these individual contributions across all data points.



Maximum Likelihood

Maximum Likelihood

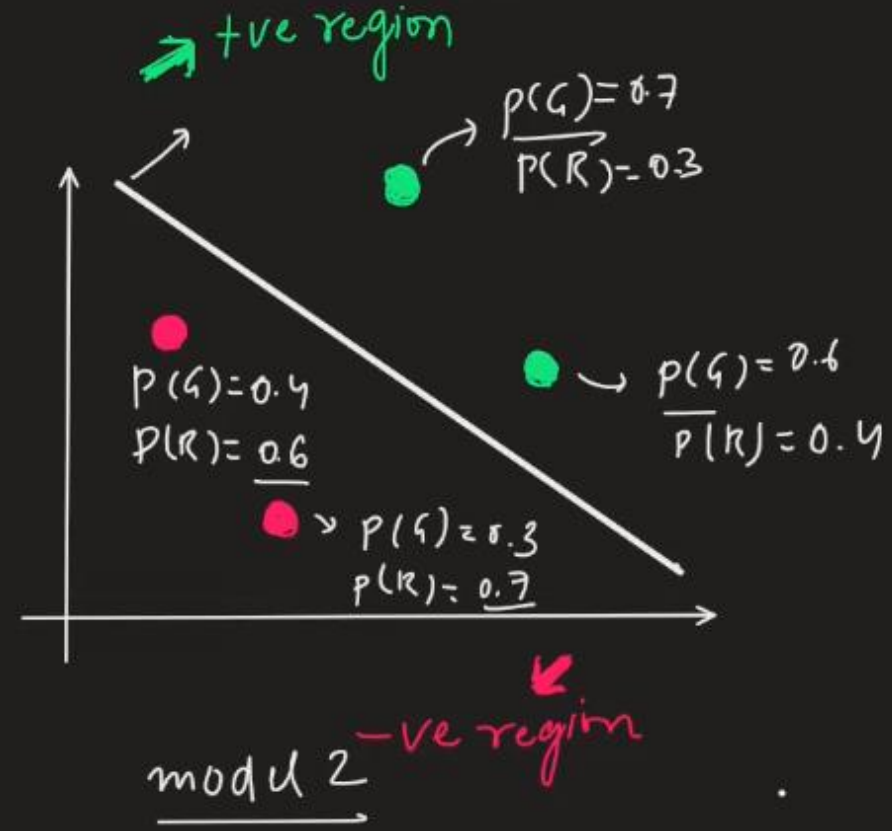
Thursday, June 17, 2021 12:19 PM



$$\hat{y} = \sigma(z)$$

$$z = \sum w_i x_i$$

model 1 $\rightarrow 0.7 \times 0.4 \times 0.4 \times 0.8 = 0.089$



model 2 $\rightarrow 0.7 \times 0.6 \times 0.6 \times 0.7 = 0.176$

Log-likelihood

1. In practice, we maximize the log-likelihood (sum of logs) to avoid numerical underflow from multiplying many small numbers.
2. Example:
 - You have 10,000 data points
 - Each prediction probability is around 0.5 on average
 - Likelihood = $0.5 \times 0.5 \times 0.5 \times \dots$ (10,000 times) = 0.5^{10000}
3. Log-Likelihood = $\log(\hat{y}_1) + \log(\hat{y}_2) + \log(\hat{y}_3) + \dots$
4. Log-Likelihood = $-0.223 + (-0.357) + (-0.357) = -0.937$
5. Cross-Entropy is just negative Log-Likelihood:
 - Cross-Entropy = $-\log(\hat{y}_1) - \log(\hat{y}_2) - \log(\hat{y}_3) - \log(\hat{y}_4) - \dots$
 - Final Cross-Entropy = $-\left[y_1 \times \log(\hat{y}_1) + (1 - y_1) \times \log(1 - \hat{y}_1)\right]$

Log Loss Error/ Binary Cross Entropy

1. The Final Step: Average Over All Data
2. For your entire dataset of N data points, the total Log Loss (Binary Cross-Entropy) is the average of all the individual losses:
 - $\text{Log Loss} = -(1/N) \times \sum [y_i \times \log(\hat{y}_i) + (1 - y_i) \times \log(1 - \hat{y}_i)]$